

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

Q1

Classes are blueprints for creating objects. They support OOP concepts like encapsulation, inheritance, polymorphism, and abstraction. They help in organizing code into reusable and modular structures.

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q2
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
    def display(self):
        return self.balance

a = BankAccount(1000)
a.deposit(500)
print("Balance after deposit:", a.balance)
a.withdraw(300)
print("Balance after withdraw:", a.balance)

print("Current Balance:", a.display())
```

```
Balance after deposit: 1500
Balance after withdraw: 1200
Current Balance: 1200
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q3
class Chapter:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

class Book:
    def __init__(self):
        self.chapters = [
            Chapter("Ch1", 10),
            Chapter("Ch2", 20),
            Chapter("Ch3", 30)
        ]
    def total_pages(self):
        return sum(c.pages for c in self.chapters)

b = Book()
print(f'Total Pages: {b.total_pages()}')
```

```
Total Pages: 60
```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

Q4

Public attributes are accessible everywhere, protected attributes use `_name`, private attributes use `__name`. Python enforces access control using naming conventions and name mangling.

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q5
class Time:
    def __init__(self, time_str):
        self.time_str = time_str

    def is_valid(self):
        try:
            h, m, s = map(int, self.time_str.split(":"))
```

```

        return 0 <= h < 24 and 0 <= m < 60 and 0 <= s < 60
    except:
        return False

    def convert(self):
        if not self.is_valid():
            return "Invalid Time"

        h, m, s = map(int, self.time_str.split(":"))
        period = "AM"

        if h >= 12:
            period = "PM"
        if h > 12:
            h -= 12
        if h == 0:
            h = 12

        return f"{h:02}:{m:02}:{s:02} {period}"

t = Time("14:25:10")
print(f'Converted Time: {t.convert()}')

```

Converted Time: 02:25:10 PM

```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q6
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount

    def display(self):
        return self.__balance

a = BankAccount(1500)
a.deposit(500)
a.withdraw(450)
print("Balance:", a.display())

```

Balance: 1550

```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q7
import random

class Card:
    def __init__(self, suit, value):
        self.suit = suit
        self.value = value

class Deck:
    def __init__(self):
        suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
        values = list(range(1, 14))
        self.cards = [Card(s, v) for s in suits for v in values]

    def shuffle(self):
        random.shuffle(self.cards)

    def deal(self, players, count):
        hands = {p: [] for p in players}
        for _ in range(count):
            for p in players:
                hands[p].append(self.cards.pop())
        return hands

players = ["A", "B", "C"]
deck = Deck()
deck.shuffle()
hands = deck.deal(players, 5)

```

```
for p in hands:
    print(p, [(c.suit, c.value) for c in hands[p]])
```

```
A [('Hearts', 11), ('Clubs', 7), ('Spades', 3), ('Spades', 13), ('Spades', 1)]
B [('Hearts', 10), ('Clubs', 11), ('Diamonds', 8), ('Clubs', 12), ('Hearts', 4)]
C [('Clubs', 6), ('Diamonds', 11), ('Spades', 8), ('Clubs', 3), ('Diamonds', 7)]
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q8
class Vehicle:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def display_info(self):
        print(self.make, self.model)

class Car(Vehicle):
    def __init__(self, make, model, num_doors):
        super().__init__(make, model)
        self.num_doors = num_doors

    def display_info(self):
        print(f"{self.make} {self.model} \nDoors: {self.num_doors}")
c = Car("Toyota", "Camry", 4)
c.display_info()
```

```
Toyota Camry
Doors: 4
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q9
class Shape:
    def area(self):
        pass
class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r
class Rectangle(Shape):
    def __init__(self, l, b):
        self.l = l
        self.b = b
    def area(self):
        return self.l * self.b

s = [Circle(5), Rectangle(4, 6)]
for i in s:
    print(f'Area: {i.area()}')
```

```
Area: 78.5
Area: 24
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q10
class CommissionEmployee:
    def __init__(self, name, sales):
        if sales < 0:
            raise ValueError("Invalid sales")
        self.name = name
        self.sales = sales
    def earnings(self):
        return self.sales * 0.1
    def __repr__(self):
        return f"{self.name} {self.sales}"

e = CommissionEmployee("John", 1000)
print(f' Earnings: {e.earnings()}')
e.sales = 2000
print(f' Earnings: {e.earnings()}')
```

```
Earnings: 100.0
Earnings: 200.0
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q11
```

```
'''Duck typing means an object is used based on its behavior (methods it has) rather than its type.'''
class Dog:
    def speak(self):
        return "Bark"
class Robot:
    def speak(self):
        return "Beep"
def describe(obj):
    print(f'{obj.__class__.__name__}: {obj.speak()}')

d = Dog()
r = Robot()
describe(d)
describe(r)
```

Dog: Bark
Robot: Beep

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q12
class Complex:
    def __init__(self, r, i):
        self.r = r
        self.i = i
    def __add__(self, other):
        return Complex(self.r + other.r, self.i + other.i)
    def __str__(self):
        return f"{self.r}+{self.i}i"

c1 = Complex(2, 3)
c2 = Complex(4, 5)
print(f'Sum: {c1 + c2}')
```

Sum: 6+8i

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q13
class InsufficientFunds(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
    def __str__(self):
        return f"Balance: {self.balance}, \nWithdrawal: {self.amount}"
def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFunds(balance, amount)
    return balance - amount
try:
    withdraw(1000, 1500)
except InsufficientFunds as e:
    print(f'Error: {e}')
```

Error: Balance: 1000,
Withdrawal: 1500

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q14
import random
class Card:
    def __init__(self, suit, rank):
        self.suit = suit
        self.rank = rank
suits = ["Hearts", "Diamonds", "Clubs", "Spades"]
ranks = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"]
deck = [Card(s, r) for s in suits for r in ranks]
random.shuffle(deck)
hand = [deck.pop() for _ in range(5)]
for card in hand:
    print(f"{card.rank} of {card.suit}")

print(f'Remaining cards in deck: {len(deck)}')
```

J of Hearts
K of Spades
6 of Clubs
7 of Diamonds
7 of Spades
Remaining cards in deck: 47

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q15
```

```
'''Data classes are more flexible than named tuples because they allow mutability, default values, type annotations, methods, and easy customization. Named tuples are immutable and limited in functionality.'''
```

```
from dataclasses import dataclass
from collections import namedtuple
```

```
@dataclass
class Student:
    name: str
    age: int = 18
    def display(self):
        return f'{self.name} {self.age}'
```

```
StudentTuple = namedtuple("StudentTuple", ["name", "age"])
s1 = Student("Amit")
s1.age = 20
print(f'Student: {s1.display()}')
s2 = StudentTuple("Dev", 22)
print(f'Student Tuple: {s2.name}, {s2.age}')
```

```
Student: Amit 20
Student Tuple: Dev, 22
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q16
```

```
import doctest
def add(a, b):
```

```
    """
    >>> add(2, 3)
    5
    >>> add(-1, 1)
    0
    >>> add(0, 0)
    0
    """
```

```
    return a + b
if __name__ == "__main__":
    doctest.testmod()
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

Q17

Output: Instance species: Instance Species Class species: Class Species Global species: Global Species Explanation: Instance namespace → self.species → "Instance Species" Class namespace → Animal.species → "Class Species" Global namespace → species → "Global Species"

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q18
```

```
import pandas as pd
import matplotlib.pyplot as plt
celsius = [0, 10, 20, 30, 40, 50]
to_kelvin = lambda c: c + 273.15
kelvin = list(map(to_kelvin, celsius))
df = pd.DataFrame({"Celsius": celsius, "Kelvin": kelvin})
print(df)
plt.plot(df["Celsius"], df["Kelvin"])
plt.xlabel("Celsius")
plt.ylabel("Kelvin")
plt.title("Celsius to Kelvin Conversion")
plt.show()
```

	Celsius	Kelvin
0	0	273.15
1	10	283.15
2	20	293.15
3	30	303.15
4	40	313.15
5	50	323.15

